

LAPORAN PRAKTIKUM ASD



Muhammad Saum Saifulloh

254107060155

Sistem Informasi Bisnis / 1E

JOBSHEET 15

Best Practice Collection

Tujuan Praktikum

1. Memahami manfaat Java Collection Framework.
2. Mengimplementasikan class dalam Java Collection Framework sesuai kasus.

Class Customer

The image shows a screenshot of a code editor, likely IntelliJ IDEA, with a Java class named 'Person' open. The class has three private attributes: 'id' (int), 'name' (String), and 'salary' (int). It includes six methods: 'getId()', 'getName()', 'getSalary()', 'setId()', 'setName()', and 'setSalary()'. The IDE interface includes a menu bar at the top (File, Edit, Selection, View, Go, Run, etc.), a toolbar with icons for various actions, and a project explorer on the left showing a project named 'person' with a package 'com.example'. The Windows taskbar is visible at the bottom, showing the Start button, a search bar, and several open applications. The system tray in the bottom right corner displays the date and time as 14:37 on 04/06/2025, along with system icons for network, volume, and battery.

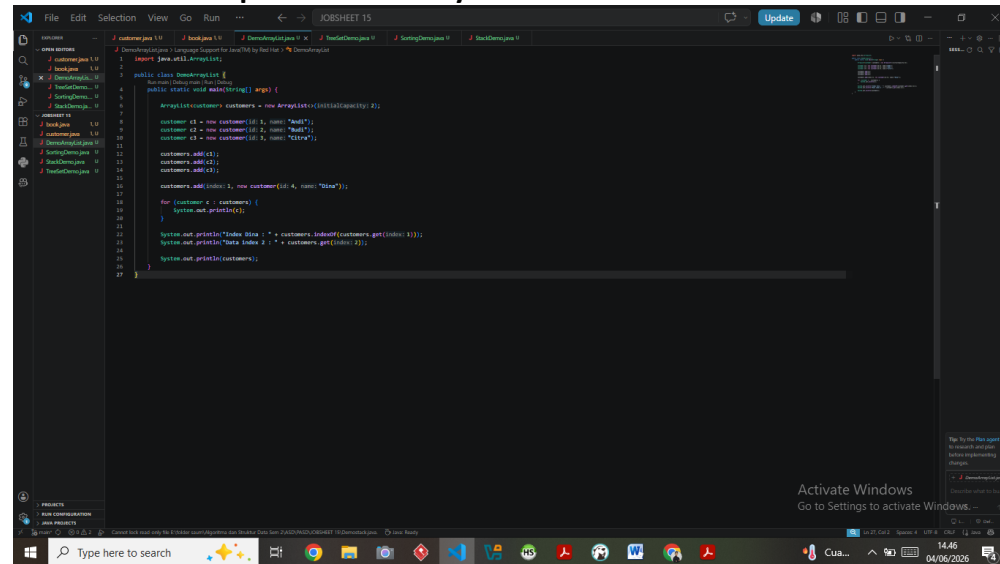
Class book

The image shows a screenshot of an IDE, likely IntelliJ IDEA, with a dark theme. The main editor window displays Java code for a class hierarchy. The code defines a base class 'Book' with attributes 'title' and 'year', and methods 'getTitle()', 'getYear()', and 'setYear()'. There are two subclasses: 'Novel' and 'ScienceFiction', both inheriting from 'Book'. 'Novel' has an additional attribute 'author' and a 'getAuthor()' method. 'ScienceFiction' has an additional attribute 'genre' and a 'getGenre()' method. The code is as follows:

```
1 public class Book {
2     public String title;
3     public String title;
4
5     public Book() {
6
7     }
8
9     public Book(String title, String year) {
10         this.title = title;
11         this.year = year;
12     }
13
14     public String getTitle() {
15         return "Title: " + this.title + " Title: " + this.title;
16     }
17
18     public String getYear() {
19         return "Year: " + this.year;
20     }
21
22     public void setYear(int year) {
23         this.year = year;
24     }
25 }
```

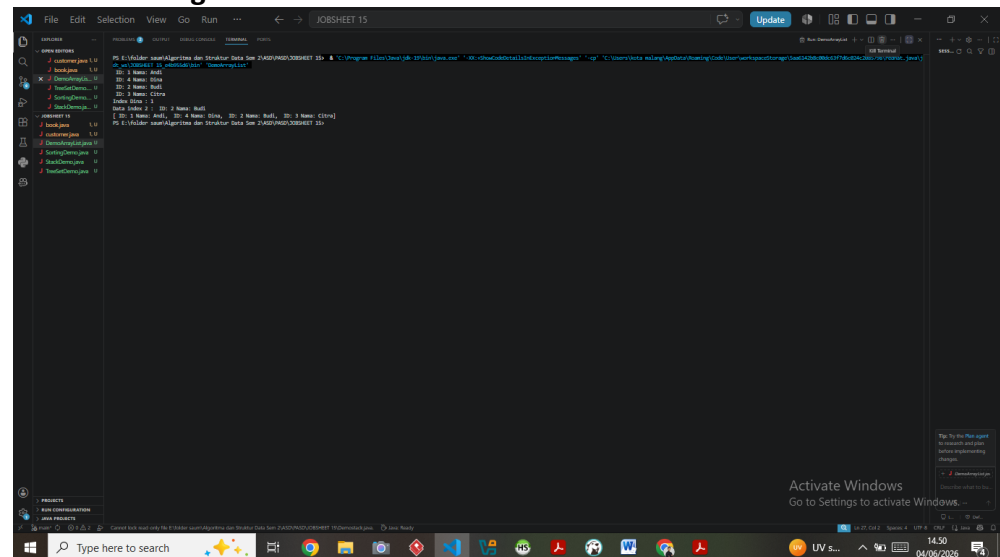
The code is written in a Java file named 'Book.java'. The IDE interface includes a menu bar at the top with options like File, Edit, Selection, View, Go, Run, and a toolbar with icons for various actions. The bottom status bar shows the current file path and some system information. The Windows taskbar is visible at the very bottom of the screen.

Praktikum 1 – Implementasi ArrayList



```
1 // DemoArrayList.java - Language Support for Java/JS by Red Hat - DemoArrayList
2
3 import java.util.ArrayList;
4
5 public class DemoArrayList {
6     public static void main(String[] args) {
7         ArrayList<Customer> customers = new ArrayList<>(initialCapacity * 2);
8
9         Customer c1 = new Customer(1, "John", "Smith");
10        Customer c2 = new Customer(2, "Jane", "Doe");
11        Customer c3 = new Customer(3, "Alice", "Johnson");
12
13        customers.add(c1);
14        customers.add(c2);
15        customers.add(c3);
16
17        for (Customer c : customers) {
18            System.out.println(c);
19        }
20
21        System.out.println("Index 0: " + customers.indexOf(customers.get(0)));
22        System.out.println("Index 1: " + customers.get(1));
23        System.out.println("Index 2: " + customers.get(2));
24
25        System.out.println(customers);
26    }
27 }
```

Hasil Run Program

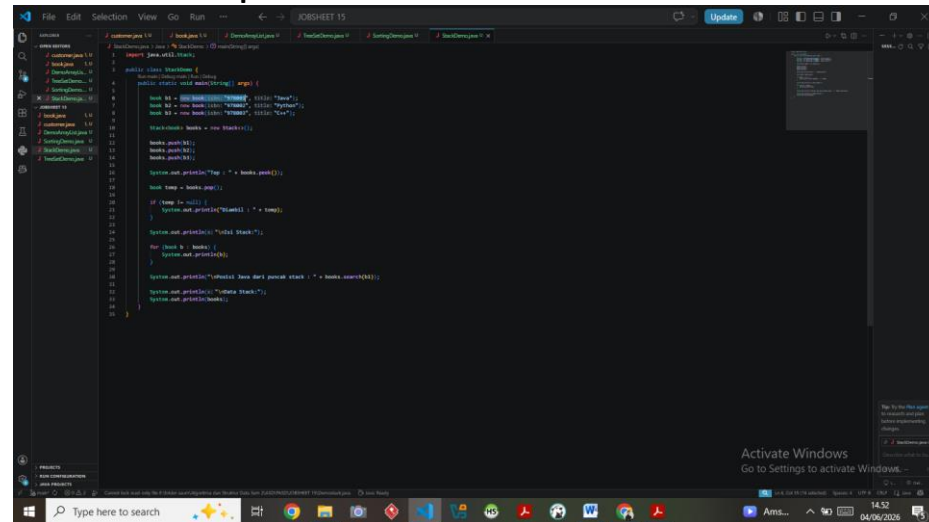


```
PS E:\Vektor> java -cp . DemoArrayList
[1] 1: John Smith, [2] 2: Jane Doe, [3] 3: Alice Johnson
PS E:\Vektor>
```

Penjelasan

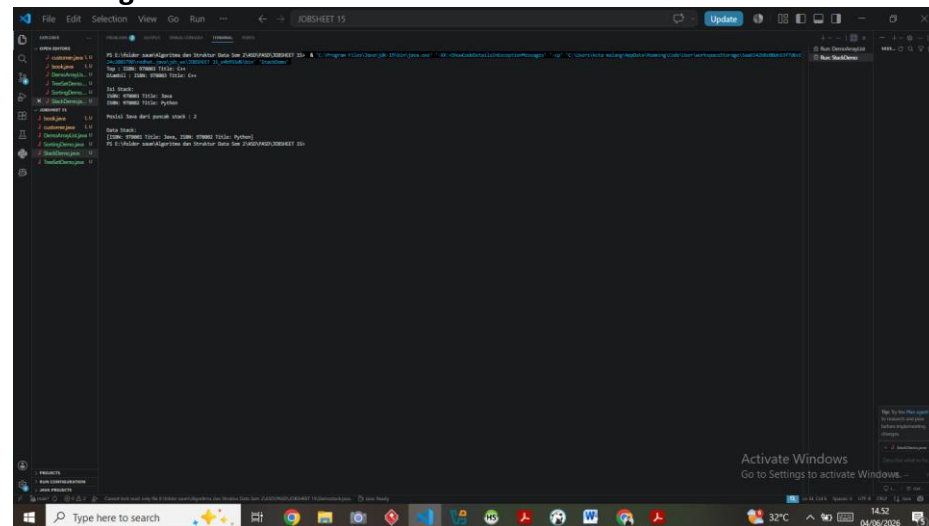
ArrayList merupakan struktur data yang memiliki ukuran dinamis sehingga jumlah data dapat bertambah secara otomatis. Method `add()` digunakan untuk menambahkan data, `add(index, obj)` untuk menyisipkan data pada indeks tertentu, `indexOf()` untuk mencari posisi objek, dan `get()` untuk mengambil data berdasarkan indeks. Indeks ArrayList dimulai dari 0. Kapasitas awal yang ditentukan saat pembuatan ArrayList tidak membatasi jumlah data yang dapat disimpan karena ArrayList akan melakukan resize secara otomatis.

Praktikum 2 – Implementasi Stack



```
1 // Stack.java
2 // Implementasi Stack menggunakan Array
3
4 import java.util.Scanner;
5
6 public class Stack {
7     private static int[] array;
8     private static int top;
9
10    public Stack(int size) {
11        array = new int[size];
12        top = -1;
13    }
14
15    public void push(int data) {
16        if (top == array.length - 1) {
17            System.out.println("Stack penuh");
18        } else {
19            array[++top] = data;
20        }
21    }
22
23    public int pop() {
24        if (top == -1) {
25            System.out.println("Stack kosong");
26            return -1;
27        } else {
28            return array[top--];
29        }
30    }
31
32    public int peek() {
33        if (top == -1) {
34            System.out.println("Stack kosong");
35            return -1;
36        } else {
37            return array[top];
38        }
39    }
40
41    public int search(int data) {
42        for (int i = 0; i < array.length; i++) {
43            if (array[i] == data) {
44                return i;
45            }
46        }
47        return -1;
48    }
49
50    public boolean isEmpty() {
51        return top == -1;
52    }
53
54    public static void main(String[] args) {
55        Scanner scanner = new Scanner(System.in);
56        Stack stack = new Stack(10);
57
58        while (true) {
59            System.out.println("Menu:");
60            System.out.println("1. Push");
61            System.out.println("2. Pop");
62            System.out.println("3. Peek");
63            System.out.println("4. Search");
64            System.out.println("5. Exit");
65
66            int choice = scanner.nextInt();
67
68            switch (choice) {
69                case 1:
69                    System.out.println("Masukkan data:");
69                    int data = scanner.nextInt();
69                    stack.push(data);
69                    break;
69                case 2:
69                    int popped = stack.pop();
69                    if (popped != -1) {
69                        System.out.println("Data yang dihapus: " + popped);
69                    }
69                    break;
69                case 3:
69                    int peeked = stack.peek();
69                    if (peeked != -1) {
69                        System.out.println("Data di puncak stack: " + peeked);
69                    }
69                    break;
69                case 4:
69                    System.out.println("Masukkan data untuk dicari:");
69                    int searchData = scanner.nextInt();
69                    int index = stack.search(searchData);
69                    if (index != -1) {
69                        System.out.println("Data ditemukan di index " + index);
69                    } else {
69                        System.out.println("Data tidak ditemukan");
69                    }
69                    break;
69                case 5:
69                    System.out.println("Program selesai");
69                    break;
69            }
69        }
69    }
69}
```

Run Program

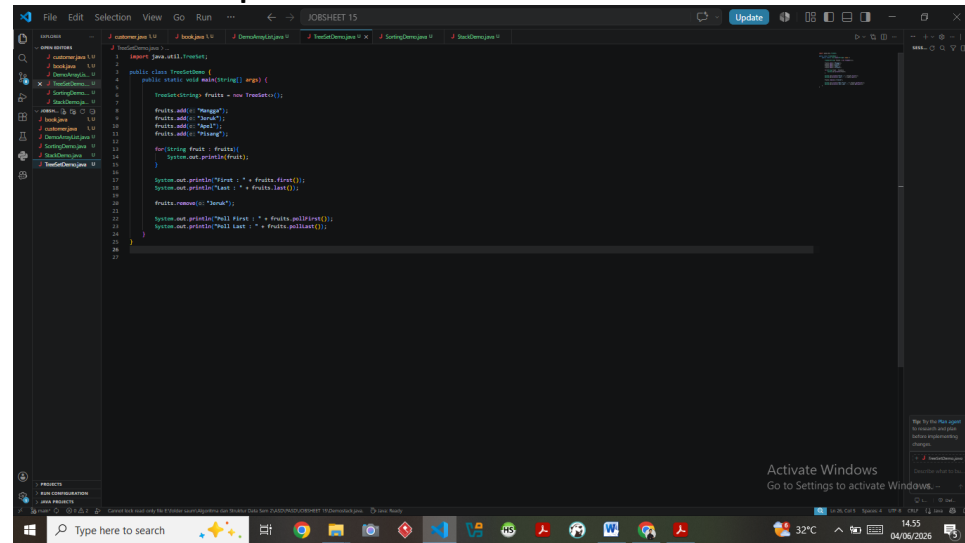


```
PS E:\Makul\sewa\algoritma\deret\deret\Stack> java Stack
Menu:
1. Push
2. Pop
3. Peek
4. Search
5. Exit
1
Masukkan data:
10
Menu:
1. Push
2. Pop
3. Peek
4. Search
5. Exit
2
Data yang dihapus: 10
Menu:
1. Push
2. Pop
3. Peek
4. Search
5. Exit
3
Data di puncak stack: 20
Menu:
1. Push
2. Pop
3. Peek
4. Search
5. Exit
4
Masukkan data untuk dicari:
10
Data ditemukan di index 1
Menu:
1. Push
2. Pop
3. Peek
4. Search
5. Exit
5
Program selesai
```

Penjelasan

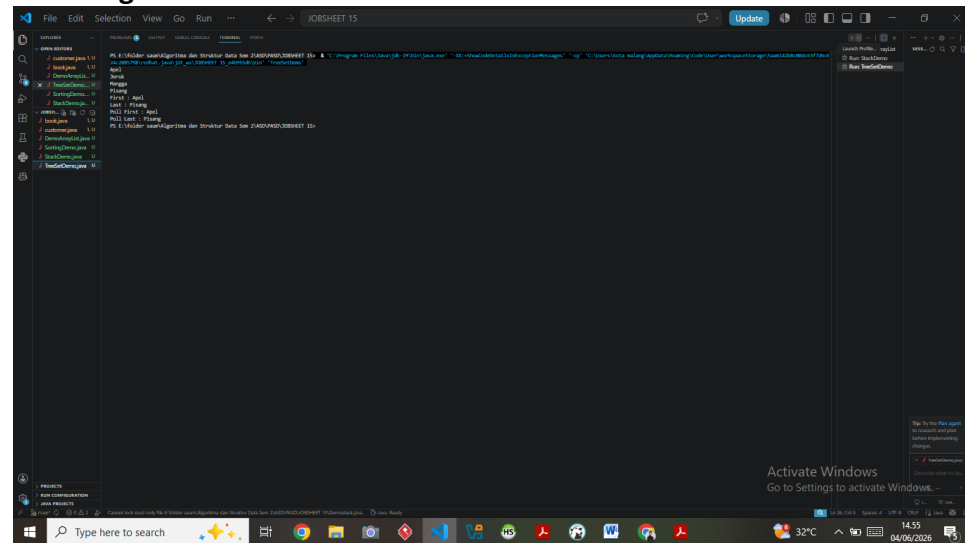
Stack merupakan struktur data yang menerapkan prinsip LIFO (Last In First Out). Method `push()` digunakan untuk menambahkan data ke puncak stack, `pop()` untuk mengambil sekaligus menghapus data teratas, dan `peek()` untuk melihat data teratas tanpa menghapusnya. Pengecekan `temp != null` dilakukan untuk menghindari kesalahan ketika data yang diambil kosong. Method `search()` digunakan untuk mencari posisi elemen dari puncak stack.

Praktikum 3 – Implementasi TreeSet



```
1 import java.util.TreeSet;
2
3 public class TreeSetDemo {
4     public static void main(String[] args) {
5         TreeSet<String> fruits = new TreeSet<>();
6
7         fruits.add("Mangga");
8         fruits.add("Pisang");
9         fruits.add("Apel");
10        fruits.add("Pisang");
11
12        for (String fruit : fruits) {
13            System.out.println(fruit);
14        }
15
16        System.out.println("First : " + fruits.first());
17        System.out.println("Last : " + fruits.last());
18
19        fruits.remove("Pisang");
20
21        System.out.println("Poll First : " + fruits.pollFirst());
22        System.out.println("Poll Last : " + fruits.pollLast());
23    }
24 }
```

Run Program

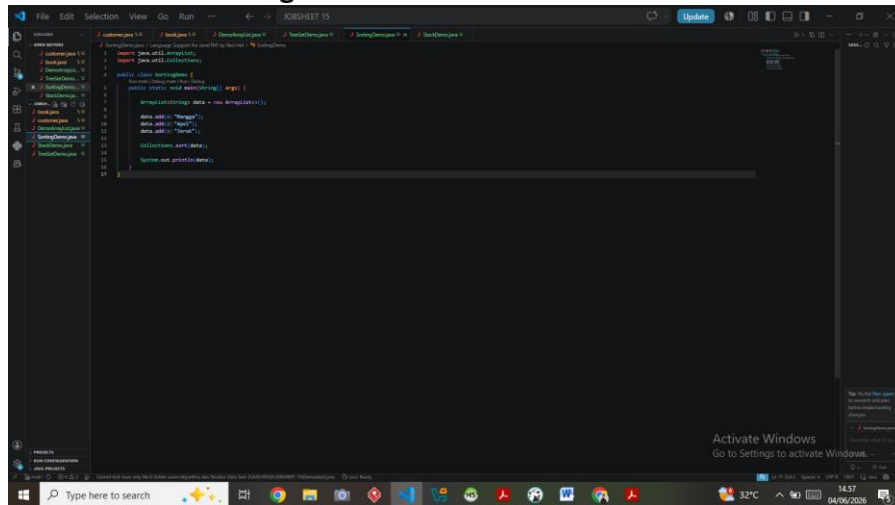


```
MS-Bindows: KonsolAlgoritma dan Struktur Data dan JAVADUNIA>JOBSEET 15>
C:\Program Files\Java\jdk-17\bin>java -cp %CLASSPATH% TreeSetDemo
Apel
Mangga
Pisang
Poll First: Apel
Poll Last: Pisang
MS-Bindows: KonsolAlgoritma dan Struktur Data dan JAVADUNIA>JOBSEET 15>
```

Penjelasan

TreeSet digunakan untuk menyimpan data yang selalu terurut secara otomatis berdasarkan urutan natural (ascending). Oleh karena itu urutan data yang ditampilkan berbeda dengan urutan saat data dimasukkan. Method `first()` digunakan untuk mengambil elemen pertama, `last()` untuk mengambil elemen terakhir, `remove()` untuk menghapus data tertentu, `pollFirst()` untuk mengambil dan menghapus elemen pertama, serta `pollLast()` untuk mengambil dan menghapus elemen terakhir.

Praktikum 4 – Sorting



```
import java.util.*;
import java.util.Collections;
import java.util.Scanner;

class Data {
    String nama;
    int umur;
}

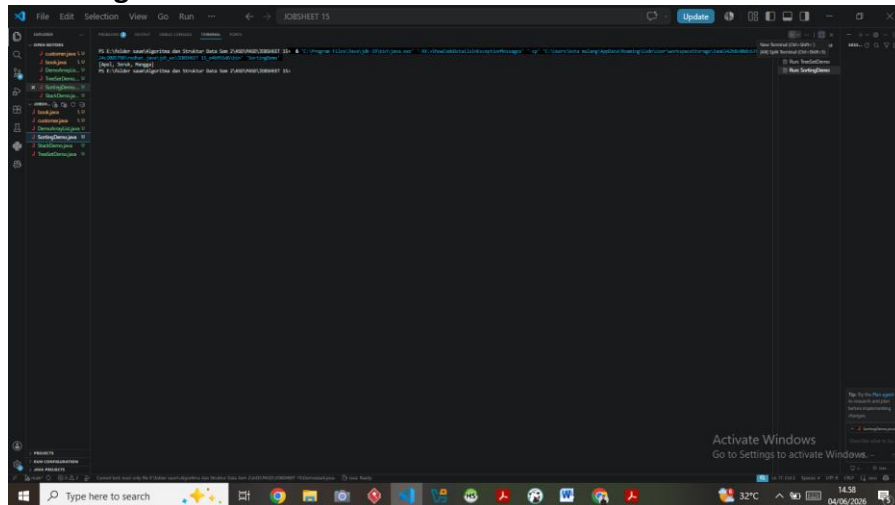
public class Sorting {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Data> data = new ArrayList<>();

        // Input data
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            String[] parts = line.split(",");
            Data obj = new Data();
            obj.nama = parts[0];
            obj.umur = Integer.parseInt(parts[1]);
            data.add(obj);
        }

        // Sorting data
        Collections.sort(data);

        // Output data
        for (Data obj : data) {
            System.out.println(obj.nama + " " + obj.umur);
        }
    }
}
```

Run Program



Penjelasan

Method `Collections.sort()` digunakan untuk mengurutkan data secara ascending. Untuk tipe data objek, pengurutan dapat dilakukan menggunakan comparator berdasarkan atribut tertentu, misalnya nama customer.

LinkGitHub: <https://github.com/saumsaifulloh/AlgoritmaDanStrukturDataSem2/tree/main/ASD/PASD/JOBSHEET%2015>